Real-world — a FastAPI API + a form page — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: the JSON API shape a real frontend calls — typed input, automatic validation, free docs — with a tiny HTML form on top. This is Doc12's FastAPI, standalone. **If not:** a frontend (or another service) has nothing clean to call, and you hand-roll validation a Pydantic model does for free.

The FastAPI service

```
# practice/app_fastapi.py   —   run:  uvicorn app_fastapi:app --reload
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from pydantic import BaseModel

from pipeline import classify

app = FastAPI(title="Sentiment")

FORM = """
<!doctype html>
<form method="post" action="/classify">
  <input name="text" placeholder="Type something">
  <button>Classify</button>
</form>
"""


class ClassifyRequest(BaseModel):
    text: str


@app.get("/", response_class=HTMLResponse)
def form():
    return FORM


@app.post("/classify")
```

```
def classify_route(body: ClassifyRequest):
    return {"label": classify(body.text)}
```

How it works

- **`ClassifyRequest` validates the body** before your function runs. A request missing `text`, or with `text` of the wrong type, is rejected with a `422` and a clear message — you never wrote that check. That's Doc04/05's "shape during", at the HTTP edge.
- **`POST /classify`** returns a plain dict, which FastAPI serializes to JSON — the shape a React app, a mobile app, or another backend consumes.
- **`GET /`** returns HTML, so a human has a form too — FastAPI can serve pages, even though its sweet spot is APIs.
- **Free docs:** open `/docs` and FastAPI has generated an interactive, tested API surface from your types — nothing to write.
- **Async when it matters:** real LLM calls spend their time waiting, so `async def classify_route(...)` + `await your_pipeline(...)` lets one worker handle many at once (Doc08b's event loop). Here the stub is instant, so sync is fine.

Testing it without a server

```
# practice/test_fastapi.py   —   python -m practice.test_fastapi
from fastapi.testclient import TestClient

from app_fastapi import app


def test_api_and_form() -> None:
    client = TestClient(app)
    html = client.get("/")
    api = client.post("/classify", json={"text": "amazing"}).json()
    missing = client.post("/classify", json={}).status_code
    result = "OK"
    if html.status_code != 200 or "form" not in html.text:
        result = "WRONG"
    if api != {"label": "positive"} or missing != 422:
        result = "WRONG"
    print(f"case 1 (form + api + validation) -> {result}")


if __name__ == "__main__":
    test_api_and_form()
```

Expected: `GET /` returns the HTML form (200); `POST /classify {"text": "amazing"}` returns `{"label": "positive"}`; a body missing `text` returns `422` (Pydantic's validation, not your code).

Why FastAPI: when the caller is **code** — a frontend, a service — a typed JSON API with automatic validation and free docs is exactly right, and the async support is what makes it handle slow LLM calls well.